Distance Vector Routing Algorithm for obtaining routing tables at each node

Aim: To write a C program to implement Distance Vector Routing Algorithm for obtaining routing tables at each node.

Description:

The **Distance Vector Routing (DVR) algorithm** is a decentralized routing protocol used in computer networks, where each router determines the best path to each destination based on distance metrics (such as hop count or delay). In this algorithm, each router maintains a table (called a "distance vector") that holds the minimum distance to every other router in the network along with the "next hop" router to use for each destination.

How It Works

1. **Distance Vectors**: Each router has a distance vector that records the shortest known distance from itself to each router in the network.
2. **Neighbor Communication**: Periodically, routers share their distance vectors only with their immediate neighbors, rather than with all routers in the network.
3. **Updating Routes**: When a router receives a distance vector from a neighboring router, it checks if any of the routes through the neighbor provide a shorter path to a particular destination:
    - If a shorter path is found, the router updates its distance vector with this new information, recording the distance and the next hop as the neighboring router.
    - This process is based on the **Bellman-Ford algorithm**, which iteratively improves the estimates of shortest paths.
4. **Convergence**: The routers continue to exchange distance vectors until all routers have the shortest path information to every other router. This point is called **convergence**, where no further updates are needed.
5. **Periodic Updates and Triggered Updates**:
    - Routers periodically send updates to neighbors to maintain up-to-date information.
    - If a router detects a change in a link (such as a failure), it triggers an immediate update to propagate the new information quickly.

Advantages of Distance Vector Routing

- **Simple and Easy to Implement**: The DVR algorithm is straightforward, making it suitable for smaller or less complex networks.
- **Local Communication Only**: Routers only need to communicate with direct neighbors, minimizing the amount of information that needs to be shared.

Disadvantages of Distance Vector Routing

- **Count to Infinity Problem**: If a link goes down, routers may keep updating distances in a loop, slowly "counting to infinity" as they propagate erroneous paths. Solutions such as **Split Horizon** and **Poison Reverse** are used to prevent this issue.
- **Slow Convergence**: Especially in large networks, it can take time for all routers to agree on the shortest paths after a network change.

Solutions to DVR Limitations

- **Split Horizon**: Prevents a router from sending information about a route back in the direction from which it was learned.
- **Poison Reverse**: Sets the distance to infinity for routes that are being advertised back to the original source, indicating no route exists.

Use Cases

Distance Vector Routing is used in protocols like:

- **RIP (Routing Information Protocol)**: A widely used protocol in small networks, which applies the DVR algorithm with a maximum hop limit to prevent routing loops.

Program:

```
#include <stdio.h>

#define INF 999

int main()
{
   int n, i, j, k;
   int cost[10][10], dist[10][10];

   printf("Enter number of nodes: ");
   scanf("%d", &n);

   printf("Enter the cost matrix:\n");
   for (i = 0; i < n; i++)
   {
      for (j = 0; j < n; j++)
      {
         scanf("%d", &cost[i][j]);

         if (cost[i][j] == 0 && i != j)
            cost[i][j] = INF;

         dist[i][j] = cost[i][j];
      }
   }

   /* Distance Vector Algorithm */
   for (k = 0; k < n; k++)
   {
      for (i = 0; i < n; i++)
      {
         for (j = 0; j < n; j++)
         {
            if (dist[i][j] > dist[i][k] + dist[k][j])
```

```
                dist[i][j] = dist[i][k] + dist[k][j];
            }
        }
    }

    /* Display routing tables */
    for (i = 0; i < n; i++)
    {
        printf("\nRouting Table for Node %d:\n", i);

        printf("Destination\tDistance\n");

        for (j = 0; j < n; j++)
        {
            if (dist[i][j] == INF)
                printf("%d\t\tINF\n", j);
            else
                printf("%d\t\t%d\n", j, dist[i][j]);
        }
    }

    return 0;
}
```

OUTPUT: 0 means there is **no direct link** between two different nodes.

```
Enter number of nodes: 4
Enter the cost matrix:
0 2 5 0
2 0 1 4
5 1 0 2
0 4 2 0
Routing Table for Node 0:
Destination    Distance
0           0
1           2
2           3
3           5

Routing Table for Node 1:
Destination    Distance
0           2
1           0
2           1
3           3

Routing Table for Node 2:
Destination    Distance
0           3
1           1
2           0
```

3 2

Routing Table for Node 3:

Destination Distance

0 5

1 3

2 2

3 0

Viva Question:

1. What is Distance Vector Routing?

Answer: Distance Vector Routing is a routing algorithm in which each router maintains a table containing the **distance (cost) to each destination** and the **next hop** to reach that destination.

2. Why is it called Distance Vector?

Answer: It is called Distance Vector because each router maintains:

- **Distance** → cost to reach a destination.
- **Vector** → direction or next-hop router to reach that destination.
-

3. Which algorithm is used in Distance Vector Routing?

Answer: It is based on the **Bellman-Ford algorithm**.

4. What is a routing table?

Answer: A routing table contains information about how to reach different destination networks or nodes.

What is the initial distance vector?

Answer: Initially:

- Distance to itself = **0**
- Distance to directly connected nodes = **link cost**
- Distance to unknown nodes = **infinity**
-

6. What is the Bellman-Ford equation?

Answer: $D_x(y) = \min_v [C(x,v) + D_v(y)]$

It means the minimum cost from node `x` to destination `y` is found by checking all neighboring nodes `v`.

7. What is the meaning of infinity in the program?

Answer: Infinity represents that there is **no known path** between two nodes. In the C program, we commonly use a large value such as `999` to represent infinity.